2.1 Section 2 – Data types and type casting – Part 1

Scroll to begin

2.1.1 Data types and type conversions

Programs are intended to process data. No matter if it is a web shop application, a human resources management system, or a computer game, each of these programs reads, processes and stores huge amounts of data. In the previous chapter, we already learned how to declare, initiate, and modify the variables that allow these data to be stored in the context of a running program. While discussing variables, the concept of data types appeared, and it turned out that the JavaScript language is weakly typed, so among other things it allows you to change the type of data stored in one variable.

We can divide the data according to their properties. For example, you will certainly intuitively distinguish between numeric data and text data. Such classification is of course arbitrary. **Numbers** can be further divided into, for example, **integer numbers** and **real numbers**.

Distinguishing data by their types is one of the characteristic features of any programming language. Each type of data is connected with certain operations we can perform on it. Usually, there are also methods of converting data between selected types (e.g. a number can be converted so that it is saved as a string).

In JavaScript, data types are divided into **primitive** (or simple) and **complex** (or composite). Among the primitive types, we can find **numbers** and **strings of characters**, while the comp types include, for example, **arrays** and objects.

The difference between these data types is contained quite precisely in their names. The primitive types, well, are simply not complex. If you write a data of a primitive type into a variable, one particular value will be stored there. This value will be atomic, in other words, it will not be possible to extract components from it. Data of complex types, such as an array, will consist of many elements of primitive (not complex) types.

Thus, we will logically deal with primitive types first.

Before we move on to discussing data types, we need to introduce one more new concept: **literals**.

Literals are a way of noting specific values (data) in the program code. Literals are found in virtually all programming languages, including JavaScript. We used literals in the previous chapter when initializing variables.

Let's take a look at an example:

```
let year = 1990;
console.log(year); // -> 1990
console.log(1991); // -> 1991
console.log("Alice"); // -> Alice
```

In this example, we declare the variable year and immediately initiate it with the value 1990. The digits 1990, written in the code at the place of variable initialization, are a literal that represents a **number**. The value 1990 is displayed on the console using the year variable. Then we display on the console the value 1991 and "Alice", in both cases using literals (representing a **number** and a **string** respectively). In JavaScript, almost each data type has its own literal.

2.1.2 The `typeof` operator

While learning about JavaScript data types, the `typeof` **operator** may be useful. Actually, it is also useful for normal work with this language, so it would be good if you remembered it for later. We will devote one of the later chapters to **operators**, but at this point it is enough to know that an operator is a symbol or name that represents some action to be performed on the indicated arguments. For example, the `+` symbol is a two-argument operator representing summation.

The `typeof` operator just mentioned is unary (it takes only one argument) and informs us of the type of data indicated as a given argument. The argument can be either a literal or a variable – in the latter case we will be told about the type of data stored in it. The typeof operator returns a string with one of the fixed values assigned to each of the types.

All possible return values of the typeof operator are:

Output

```
"undefined"
"object"
"boolean"
"number"
"bigint"
"string"
"symbol"
"function"
```

This list roughly shows us what types of data we will be dealing with in JavaScript.

Let's test the typeof operator using a simple example:

```
let year = 1990;
console.log(typeof year); // -> number
console.log(typeof 1991); // -> number

let name = "Alice";
console.log(typeof name); // -> string
console.log(typeof "Bob"); // -> string

```

```
let typeOfYear = typeof year;
console.log(typeOfYear); // -> number
console.log(typeof typeOfYear); // -> string
```

Again we declare and initiate the variable `year`.

As you can see, `typeof` for both the literal `1991` and the variable containing a number (we initialize it with the literal `1990`) will return the word `"number"`.

We perform a similar test on the `"Alice"` and `"Bob"` strings, using the variable `name`.

Additionally, we do a small experiment. The result of `typeof` year is stored to the variable named `typeOfYear`.

As you can see, it stores the value as a `"number"`. If we check the type of this variable, we get `"string"`.

Check the example yourself in the editor.

2.1.3 Primitive data types

In JavaScript, there are six primitive (or simple) data types: **Boolean, Number, BigInt, String, Symbol**, and **undefined**. Additionally, the primitive **null** value is also treated as a separate type. The primitive, as we have already said, is a type of data whose values are atomic. This means that the value is one, indivisible element.

Let's try to take a closer look at primitives.

Boolean

The Boolean is a logical data type. It can only take one of two values: `true` or `false`. It's mainly used as a conditional expression needed for deciding what part of the code should be executed, or how long something should be repeated (this is called a control flow statement, and we'll take a closer look at it in Module 4).

Booleans are also used as what is commonly referred to as a **flag**, a variable that signals something that can be either present or absent, enabled or disabled, etc. Like any other variable, booleans should have clear and informative names. It's not mandatory, but we can often see that boolean flag names are prefixed with "is", to show the intent of checking this for true/ false values.

```
let isDataValid = true;
let isStringTooLong = false;
let isGameOver = false;
continueLoop = true;

console.log(false); // -> false
console.log(typeof false); // -> boolean
console.log(isDataValid); // -> true
console.log(typeof isDataValid); // -> boolean
```

We can perform, without conversion (i.e. change to another type) logical operations on boolean values, some perhaps that you know from mathematics, such as NOT, AND, and OR (the symbols !, && and || correspondingly). We will find out more about them in the chapter on operators.

Number

This is the main numeric type in JavaScript that represents both real numbers (e.g. fractions) and integers. The format in which the data of this type is stored in the memory means that the values of this type are sometimes approximate (especially, but not only, very large values or some fractions). It is assumed, among other things, that in order to ensure the correctness of calculations, the integer values should be limited in JavaScript to the range from `-(253 - 1)` to `(253 - 1)`.

Numbers allow for all typical arithmetic operations, like addition, subtraction, multiplication, and division.

```
const year = 1991;
let delayInSeconds = 0.00016;
let area = (16 * 3.14);
let halfArea = area / 2;

console.log(year); // -> 1991;
console.log(typeof year); // -> number;
```

Numbers in JavaScript are usually presented in decimal form, which we are used to in everyday life. However, if a literal describing a number is preceded by an appropriate prefix, we can present it in hexadecimal `(0x…)`, octal `(0o...)` or binary `(0b...)` form. We can also write numbers in exponential form, so for example, instead of `9000`, we can write `9e3`, and instead of `0.00123`, we can write `123e-5`. You are probably already familiar with the terms we used just now, such a decimal, hexadecimal, or exponential representation.

```
let a = 10; // decimal - default
let b = 0x10; // hexadecimal
let c = 0o10; // octal
let d = 0b10; // binary

console.log(a); // -> 10
console.log(b); // -> 16
console.log(c); // -> 8
console.log(d); // -> 2

let x = 9e3;
let y = 123e-5;
console.log(x); // -> 9000
console.log(y); // -> 0.00123
```

In addition to regular numbers in JavaScript, we use three additional special values, which are: `Infinity`, `-Infinity` and `NaN` (not a number). The first two do not require any additional explanations – they are exactly what we know from mathematics. The last one, `NaN`, is not so much a numerical value as a notification that some arithmetic action (or mathematical function) could not be performed because the argument is either not a number, or cannot be converted to a number.

```
let a = 1 / 0;
let b = -Infinity;

console.log(a); // -> Infinity
console.log(b); // -> -Infinity
console.log(typeof a); // -> number
console.log(typeof b); // -> number

let s = "it's definitely not a number";
let n = s * 10;
console.log(n); // -> NaN
console.log(typeof n); // -> number
```

Test these examples, and try to change the values that appear in them yourself.

BigInt

The **BigInt** type is not used too often. It allows us to write integers of virtually any length. For almost any normal numerical operations, the **Number** type is enough, but from time to time we need a type that can handle much bigger integers.

We can use mathematical operations on BigInts in the same way as on Numbers, but there is a difference when dividing. As the BigInt is an integer type, the division result will always be **rounded down** to the nearest whole number.

BigInt literals are numbers with the **...n** suffix.

```
let big = 1234567890000000000000n;
let big2 = 1n;

console.log(big); // -> 1234567890000000000000n
console.log(typeof big); // -> bigint

console.log(big2); // -> 1n
console.log(7n / 4n); // -> 1n
```

You cannot use other types in arithmetic operations on BigInts, that is, you cannot add a BigInt and a Number to each other (this will generate an error).

```
let big3 = 1000n + 20;
// -> Uncaught TypeError: Cannot mix BigInt and other types, use expli
```

The BigInt does not have its own equivalent of `Infinity` or `NaN` values. In the case of the Number type, such values appear when dividing by 0 (Infinity result) or trying to perform an arithmetic action on a value that is not a number (NaN result). In the case of the BigInt type, such actions will generate an error.

```
let big4 = 1000n / 0n; // -> Uncaught RangeError: Division by zero
```

String

The **String** type represents a sequence of characters forming a piece of text. Common ope on texts include concatenation, extraction of the substring, and checking the length of the s Strings are extensively used in programming and even more so in web development, as both HTML and a big part of Internet content is text.

The most common use of text in web development includes:

- links and paths to resources;
- tokens;
- checking user-filled forms and input;
- dynamic content generation

Strings, like other primitives, are immutable, so when we want to change even one letter in a string, in reality, we create a new string.

In previous examples, we already used character strings. We used quotation marks to indicate that a given text is to be treated as a string (i.e. String type). String literals can be created using single or double quotes, as long as both beginning and end quote characters match up.

```
let country = "Malawi";
let continent = 'Africa';

console.log(country); // -> Malawi
console.log(typeof country); // -> string
console.log(continent); // -> Africa
console.log(typeof continent); // -> string
```

If you use double quotes to mark a string, you can place single quotes inside the string, and they will be treated as ordinary characters. This will also work in the opposite situation (i.e. placing double quotes between the single quotes).

```
let message1 = "The vessel 'Mars' called at the port.";
let message2 = 'Cyclone "Cilida" to pass close to Mauritius.';

console.log(message1); // -> The vessel 'Mars' called at the port.
console.log(message2); // -> Cyclone "Cilida" to pass close to Mauriti
```

If you want to put a single or double quote inside the string, you can also use the escape character – backslash. A quote mark preceded by the \ (backslash) character will be interpreted as ordinary characters that are part of our string, not parts of a literal construction. The backslash itself, if it is to be treated as an ordinary character (not a control character), must also be preceded by ... escape character (i.e. a backslash).

```
let message1 = 'The vessel \'Mars\' called at the port.';
let message2 = "Cyclone \"Cilida\" to pass close to Mauritius.";

console.log(message1); // -> The vessel 'Mars' called at the port.
console.log(message2); // -> Cyclone "Cilida" to pass close to Mauriti

let path = "C:\\Windows";
console.log(path); // -> C:\Windows
```

Trying to perform arithmetic operations on String type values, such as subtraction, multiplication, or division, will usually end in an error. More precisely, the NaN value will be returned as a result of the action.

Why is this happening? Seeing the arithmetic operators `-`, `*`, or `\`, the JavaScript interpreter tries to interpret the given values as numbers, or convert them into numbers. So if the character strings consists of digits, the automatic conversion will be successful and we will get the result of the arithmetic action as a Number type value. If the character string cannot be interpreted as a number (and converted) we will get the NaN result. We will talk more about conversion in a moment.

```
let path = "C:\\Windows" - "Windows";
console.log(path); // -> NaN

let test = "100" - "10";
console.log(test); // -> 90
console.log(typeof test); // -> number
```

The exception is the addition operation, which will not be treated as an arithmetic one, but as an attempt to create a new string by combining two input strings.

```
let path = "C:\\" + "Windows";
```

```
console.log(path); // -> C:\Windows

let test = "100" + "10";
console.log(test); // -> 10010
console.log(typeof test); // -> string
```

A very convenient mechanism that was introduced to JavaScript in 2015 is **string interpolation**. It allows you to treat a character string as a template, in which you can place values in selected places, such as those taken from variables. Such a literal is created using backticks (or grave accents) instead of quotation marks. The places where values are inserted are marked with curly brackets preceded by a `$` sign.

```
let country = "Malawi";
let continent = "Africa";

let sentence = ` ${country} is located in ${continent}.`;
console.log(sentence); // -> Malawi is located in Africa.
```

You can do a lot of useful work on String type data. Unfortunately, they require two new concepts: **methods** (and indirectly, objects) and **autoboxing**. The exact explanation of both concepts goes beyond the scope of this course, so we will try to make them a little simpler.

In one of the previous chapters, we introduced the concept of a function, also in a somewhat simplified form. Now let's talk about methods.

A **method** is a special kind of function that belongs to an object. **Objects** are complex data types, which can consist of many values (stored in properties) and methods. If you want to call the method of an object, you write the name of the method after a dot. Does this remind you of something? This is exactly the notation you use when calling `console.log`. The console object has many other methods besides the `log` method, such as `time` and `timeEnd` (which can be used to measure time).

```
console.time();
console.log("test console"); // -> test console
console.timeEnd(); // -> default: 0.108154296875 ms
```

All data of primitive types such as Number, BigInt, Boolean, or String have corresponding objects to which they can be converted. Each of these objects will have methods designed for a specific data type. At this point, we come to another concept, that is, **autoboxing**. If a dot appears after a

literal representing a primitive type, or after a variable containing this type of data, the JavaScript interpreter tries to treat this value as an object and not a primitive. For this purpose, it converts the primitive to the corresponding object on the fly, which has the appropriate methods (i.e. it performs autoboxing). A bit confusing, isn't it? Fortunately, in order to use methods, we don't have to understand it exactly – it's enough to follow the given convention.

Let's take a look at an example:

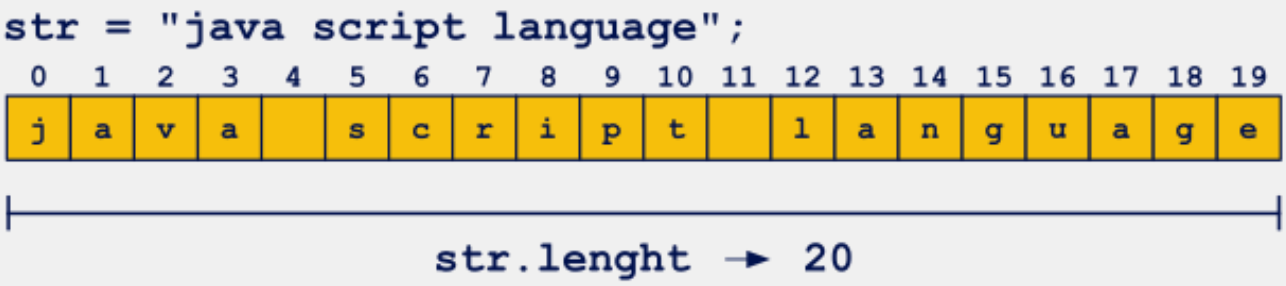
```
let river = "Mekong";
let character = river.charAt(2);
console.log(character); // -> k
```

In the variable `river`, we store the primitive of a String type. In the next line, we refer to this variable, writing a dot after its name and the name of one of the methods – `charAt` (a method of the String class object). Although the primitive has no methods that can be called, the interpreter temporarily converts this value to a suitable object that already has such methods. One of these methods is `charAt`, which we now call. The method operates on a string originally placed in the river variable, and returns a single letter from the specified position (letters are counted starting from 0).

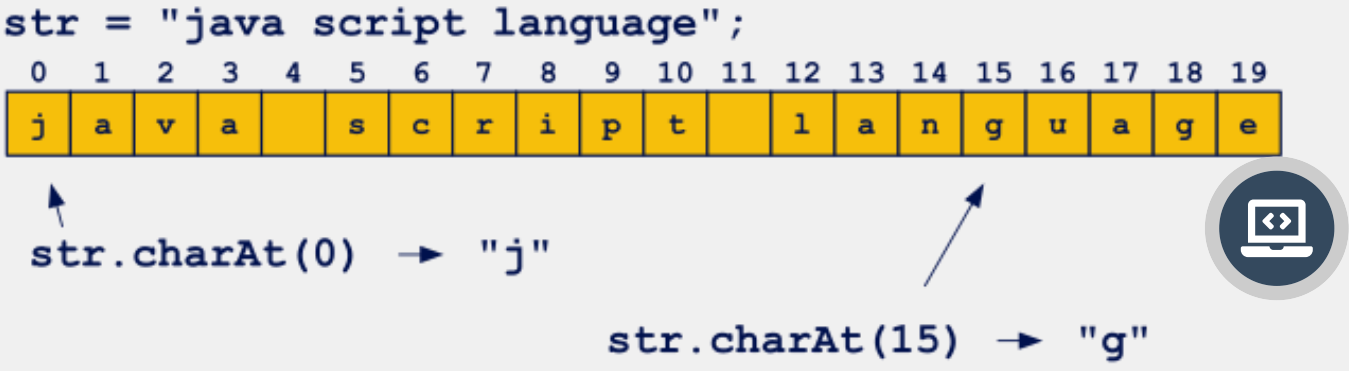
After the operation is completed, the interpreter removes the temporary object. So from our point of view, it looks like we just called a method on a given primitive type.

Commonly used string methods and properties (i.e. named values related to the object) are:

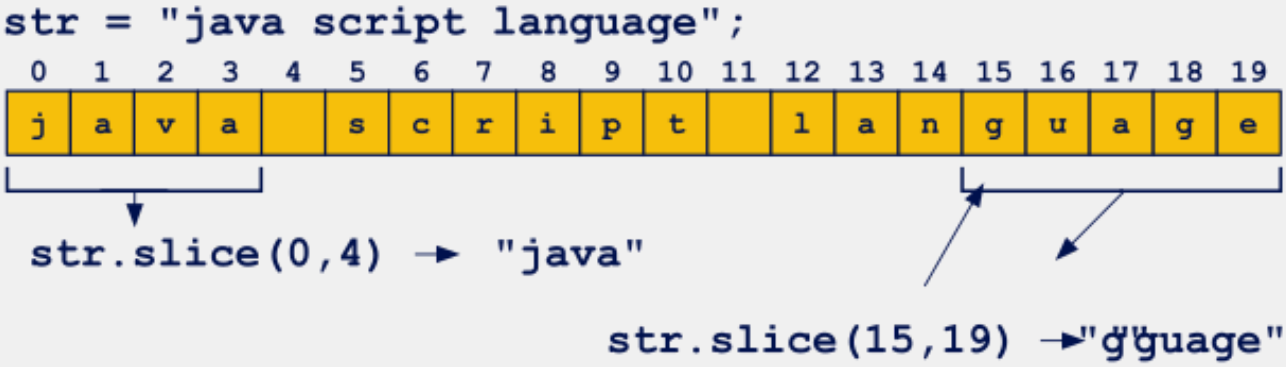
- `length`: property, returns the number of characters in a string;



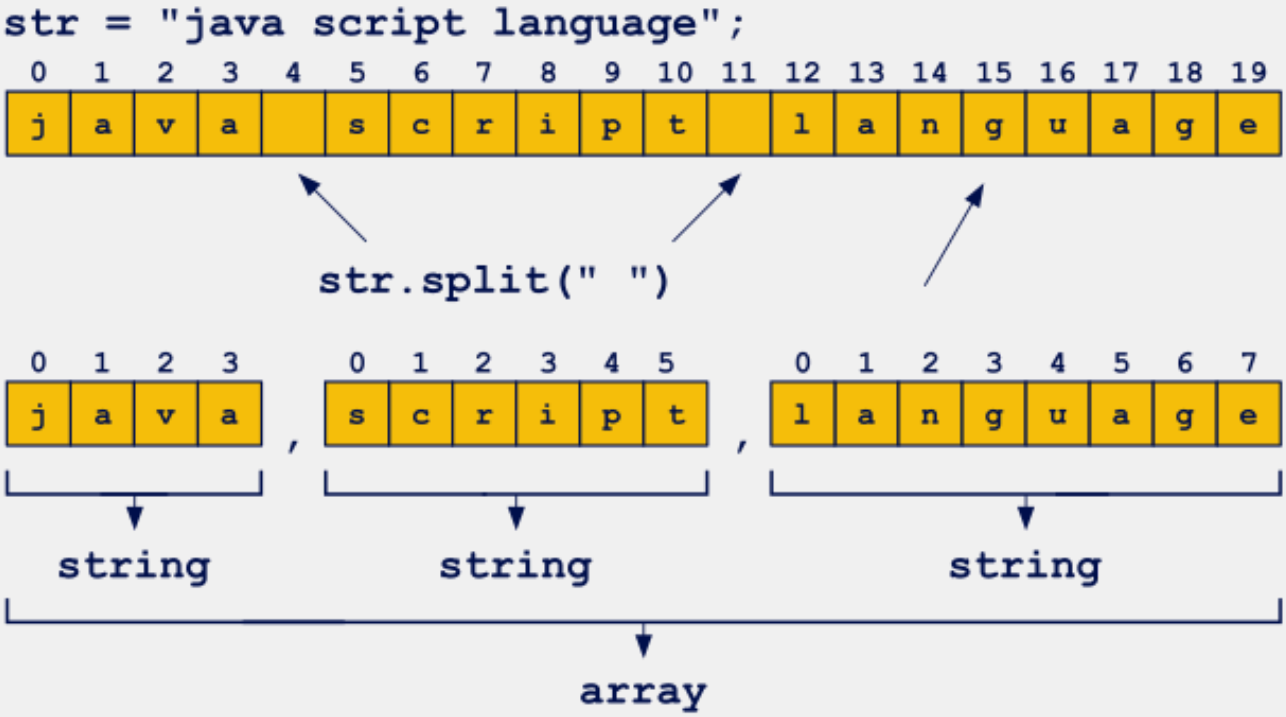
- `charAt(index)`: method, returns the character at the "index" position in the string (indexes start from 0);



- `slice(beginIndex, [optional] endIndex)`: method, returns a new string that is created from the characters between `beginIndex` (included) and `endIndex` (excluded); if `endIndex` is omitted, then the new string is from `beginIndex` to the end of the string;



- `split(separator, [optional] limit)`: method, splits the string into substrings whenever a separator is found in that string, and returns an array of those substrings (we will say a few words about arrays in a moment), while an optional `limit` limits the number of substrings added to the list.



```
let str = "java script language";

console.log(str.length); // -> 20
console.log('test'.length); // -> 4

console.log(str.charAt(0)); // -> 'j'
console.log('abc'.charAt(1)); // -> 'b'

console.log(str.slice(0, 4)); // -> 'java'
console.log('test'.slice(1, 3)); // -> 'es'

console.log(str.split(' ')); // -> ['java', 'script', 'language']
console.log('192.168.1.1'.split('.')); // -> ['192', '168', '1', '1']
```

To understand it properly, it is necessary to run examples of String type data. Do not be afraid to experiment by changing the data in the examples, adding new variables, or displaying additional information on the console.

Undefined

The undefined type has only one value: `undefined`. It's the default value that all variables have after a declaration if no value is assigned to them. You can also assign the value `undefined` to any variable, but in general, this should be avoided, because if we need to mark a variable as not holding any meaningful value, we should use `null`.

```
Let declaredVar;
console.log(typeof declaredVar); // -> undefined

declaredVar = 5;
console.log(typeof declaredVar); // -> number

declaredVar = undefined;
console.log(typeof declaredVar); // -> undefined

The undefined value can also be returned by the typeof operator when a

Console.log(typeof notDeclaredVar); // -> undefined
console.log(notDeclaredVar); // -> Uncaught ReferenceError: notDeclare
```

Symbol

The **Symbol** type is, well... complicated to say the least. And fortunately, not particularly us us.

It's a new primitive type that was added to JavaScript in 2015. It doesn't have any literal value, and can only be created using a special constructor function. Symbols are a form of identifier that are guaranteed to be unique.

Symbols are an advanced topic, and to understand their power and usefulness, we'll need to cover a lot of other topics first, so for now, just remember that the Symbol type exists.

Null

The `null` value is quite specific. The value itself is primitive, while the type to which it belongs is not a primitive type, such as Number or undefined. This is a separate category, associated with complex types, such as objects. The `null` value is used to indicate that the variable does not contain anything, and most often it is a variable that is intended to contain values of complex types.

In a nutshell, we can assume that the `undefined` value is assigned to uninitialized variables automatically, but if we want to explicitly indicate that the variable does not contain anything, we assign it a `null` value. One important caveat for `null` is that when checked with the `typeof` operator, it will return `"object"`, a surprising result. This is a part of a much more complicated object system, but for now, you just need to know that `typeof null` is equal to `"object"`.

```
let someResource;
console.log(someResource); // -> undefined
console.log(typeof someResource); // -> undefined

someResource = null;
console.log(someResource); // -> null
console.log(typeof someResource); // -> object
```

In this course, however, apart from minor mentions, we will not be learning a concept known as object-oriented programming, and therefore, using the `null` value will not be so important to us for the time being.

2.1.4 Type conversions

Primitive construction functions

Using literals is not the only way to create variables of the given primitive types. The second option is to make them using **constructor** functions. These types of functions are mainly used in JavaScript for object-oriented programming, which is outside the scope of our course. However, these few listed constructor functions can also be used to create primitives, not just objects (this is not a general feature, but only for the listed functions). The following functions will return primitives of a given type: `Boolean`, `Number`, `BigInt`, and `String`.

Most of these functions can be called without any arguments. In such a situation:

- the function `String` will by default create and return an empty string – primitive "";
- the function `Number` will by default create and return the value 0;
- the function `Boolean` will by default create and return the value of false.

The function `BigInt`, unlike other constructor functions, requires you to pass some initial value to it. This can be an integer number that will be converted to a BigInt (see examples).

```
const str = String();
const num = Number();
const bool = Boolean();

console.log(str); // ->
console.log(num); // -> 0
console.log(bool); // -> false

const big1 = BigInt(42);
console.log(big1); // -> 42n

const big2 = BigInt(); // -> Uncaught TypeError: Cannot convert undefi
```

But creating default values is not impressive at all. We can accomplish these using literals. So what do we use these functions for? Well, we use them in type conversions.

2.1.5 Conversions

It's a pretty common situation to have a value of one type but to need a value of another type. The simplest example is when we have a number, but we need to add it to some text. Conversions in JavaScript happen automatically in specific situations, but can also be used explicitly through functions like `String()` or `Number()`. Earlier we saw how those functions could be used to create default values of those types, but that's not all they can do. Those functions also accept arguments in parentheses and (if possible) will convert them to a given type.

```
Const num = 42;

const strFromNum1 = String(num);
const strFromNum2 = String(8);
const strFromBool = String(true);
const numFromStr = Number("312");
const boolFromNumber = Boolean(0);
```

Most of these conversions are straightforward, but some may be a little confusing, so let's discuss each case of primitive conversion. Test all of the examples shown for type conversion. Try to experiment with your own values.

2.1.6 Conversion to String

Conversions are the easiest to understand, as they try to directly change the value to a string, and this can be done for all primitive types. So there are no surprises there. Note that in the example, we used the recently discussed technique of character **string interpolation**.

```
let str = "text";
let strStr = String(str);
console.log(`${typeof str} : ${str}`); // -> string : text
console.log(`${typeof strStr} : ${strStr}`); // -> string : text

let nr = 42;
let strNr = String(nr);
console.log(`${typeof nr} : ${nr}`); // -> number : 42
console.log(`${typeof strNr} : ${strNr}`); // -> string : 42

let bl = true;
let strBl = String(bl);
console.log(`${typeof bl} : ${bl}`); // -> boolean : true
console.log(`${typeof strBl} : ${strBl}`); // -> string : true

let bnr = 123n;
let strBnr = String(bnr);
console.log(`${typeof bnr} : ${bnr}`); // -> bigint : 123
console.log(`${typeof strBnr} : ${strBnr}`); // -> string : 123

let un = undefined;
let strUn = String(un);
console.log(`${typeof un} : ${un}`); // -> undefined : undefined
console.log(`${typeof strUn} : ${strUn}`); // -> string : undefined

let n = null;
let strN = String(n);
console.log(`${typeof n} : ${n}`); // -> object : null
console.log(`${typeof strN} : ${strN}`); // -> string : null
```

2.1.7 Conversion to Number

Conversion to a number is not as obvious as conversion to a string. It works as expected for strings that represent actual numbers, like `"14"`, `"-72.134"`, or strings that represent numbers in scientific notation, like `"2e3"`, or special number values like `"NaN"` or `"Infinity"`.

However, the string can also contain numbers in hexadecimal, octal, and binary format. The be preceded by 0x, 0o, or 0b respectively. For any string that cannot be converted to a spe value, `NaN` (not a number) is returned. A `BigInt` can also be converted to a `Number`, but we to remember that a BigInt can store much bigger values than a Number, so for large values, part of them can be truncated or end up being imprecise. The Boolean `true` is converted to `1`, and `false` to `0` – this is common for many programming languages. An attempt to convert an undefined value will result in NaN, while null will be converted to `0`.

```
console.log(Number(42)); // -> 42

console.log(Number("11")); // -> 11
console.log(Number("0x11")); // -> 17
console.log(Number("0o11")); // -> 9
console.log(Number("0b11")); // -> 3
console.log(Number("12e3")); // -> 12000
console.log(Number("Infinity"));// -> Infinity
console.log(Number("text")); // -> NaN

console.log(Number(14n)); // -> 14
console.log(Number(123456789123456789123n)); // - > 123456789123
456800000

console.log(Number(true)); // -> 1
console.log(Number(false)); // -> 0

console.log(Number(undefined)); // -> NaN

console.log(Number(null));// -> 0
```

2.1.8 Conversion to Boolean

Conversions to Boolean follow simple rules, as we can only have one of two values: `true` or `false`. The value `false` is always returned for:

- `0`,
- `NaN`,
- empty string,
- `undefined`,
- `null`

Any other value will result in `true` being returned.

```
console.log(Boolean(true)); // -> true

console.log(Boolean(42)); // -> true
console.log(Boolean(0)); // -> false
console.log(Boolean(NaN)); // -> false

console.log(Boolean("text")); // -> true
console.log(Boolean("")); // -> false

console.log(Boolean(undefined)); // -> false

console.log(Boolean(null)); // -> false
```

2.1.9 Conversion to BigInt

In order for conversions to a BigInt to succeed, we require a Number or String representing a number as a value to be converted. Values for conversion can be given in the default decimal

form, as well as in hexadecimal, octal, or binary form. This applies both to the situation where we give the Number and String literals as arguments (or variables containing data of those types). We can also use exponential notation, but only for Number arguments. Unlike other conversions, conversion to a BigInt will throw an error, and will stop the program when unable to convert a given value.

Note: When testing the following example, please pay attention to the fact that the first error prevents further code execution. So run the example several times in succession, removing the wrong calls one by one.

```
console.log(BigInt(11)); // -> 11n
console.log(BigInt(0x11)); // -> 17n
console.log(BigInt(11e2)); // -> 1100n

console.log(BigInt(true)); // -> 1n

console.log(BigInt("11")); // -> 11n
console.log(BigInt("0x11")); // -> 17n

console.log(BigInt(null)); // -> Uncaught TypeError: Cannot convert nu

console.log(BigInt(undefined)); // -> Uncaught TypeError: Cannot conve

console.log(BigInt(NaN)); // -> Uncaught RangeError: The number NaN ca
```

2.1.10 Implicit Conversions

Conversions can also happen automatically, and they happen all the time. This simple example will demonstrate it (we tested a similar example when discussing the String type):

```
const str1 = 42 + "1";
```

```
console.log(str1); // -> 421
console.log(typeof str1); // -> string

const str2 = 42 - "1";
console.log(str2); // -> 41
console.log(typeof str2); // -> number
```

So what's going on? The details will be shown in the chapter on operators, but the short answer is that when we try to perform an addition when one of the arguments is a string, JavaScript will convert the rest of the arguments to a string as well. This is what is happening with `str1` in the example. Subtraction with a string, however, doesn't make much sense, so in that case JavaScript converts everything to Numbers.

2.1.11 SECTION PRACTICE

Question 1: Write a code that will create variables and initialize them with values of `Boolean`, `Number`, `BigInt`, `String`, and undefined types using (when possible) literals and constructor functions.

Show Example

Question 2: Print all values and all types of those values using `console.log`. Try to use string interpolation to display the value and type at the same time with a single `console.log` call, e.g. in the following form: `1000 [number]`.

Show Example

Question 3: Carry out a chain of conversions: create a `Boolean` from a `BigInt` created from a `Number` that was created from a `String`. Start with the value `"1234"`. Is it possible?

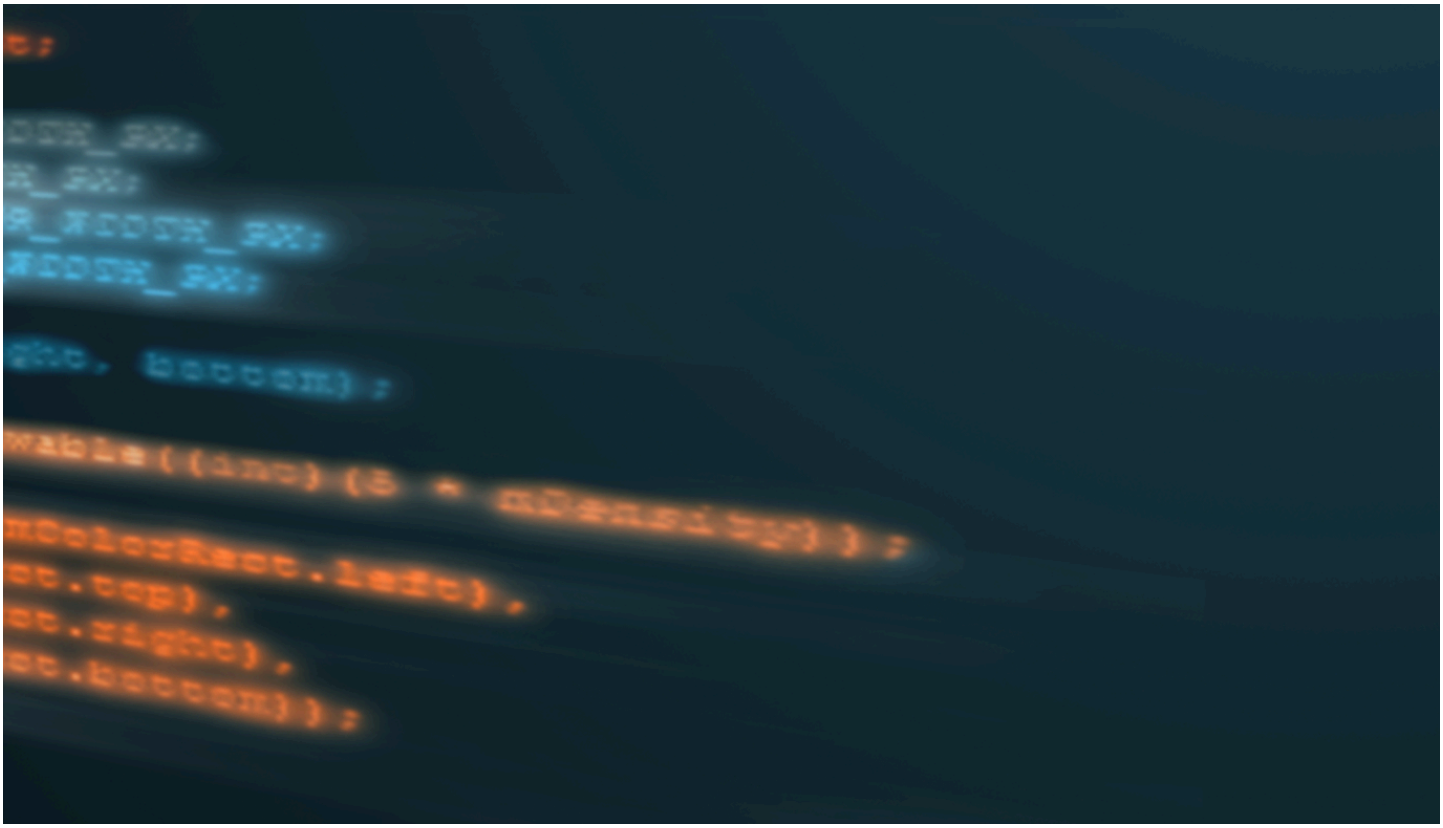
Show Example

Question 4: Try adding two values of the same type and check the result type. Try it for all primitive types.

Show Example

Question 5: Try adding two values of different types and check the results.

Show Example

Question 6: Try to modify the line `const str1 = 42 + "1";` to get the result `43` (without removing the quotes around `1`).

Show Example